

Debreceni Egyetem
Informatikai Kar
Adattudomány és Vizualizáció Tanszék

**Dinamikus objektumelkerülés megerősítéssel
a CARLA szimulátorban**

Témavezető:

Mészáros László
Tanársegéd

Készítette:

Angi Dávid
Adattudomány MSc

Debrecen, 2026

Tartalomjegyzék

1. Bevezetés	3
2. Szimulációs környezet	4
3. Programozás és algoritmus	7
3.1. Megerősítéses tanulás	7
3.2. Python programozási nyelv	7
3.3. Soft Actor-Critic SB3	7
4. Szensoradatok előfeldozása, Autóenkóderek	7
4.1. Kamera	7
4.2. LiDAR	7
4.3. Szenzorfuzio	7
5. Jutalom függvény	7
6. Betanult modellek összehasonlítása	7
7. Összegzés	7
Hivatkozások	8

1. Bevezetés

A 21. század egyik ha nem a legnagyobb hatású technológiai átalakulása a közlekedésben zajlik. Az elmúlt másfél évtizedben szemünk láttára kerültek önvezető járművek a kutatólaboratóriumokból a hétköznapi forgalom részévé. Kezdetben vezetéstámogató rendszerek (sávtartó automatika, adaptív tempomat, vészfékasszisztens) kirivó már technológiának számítottak azonban ma már a középkategóriás autókban is alapfelszereltség részét képezik, egyes városokban pedig sofőr nélküli robottaxik szállítják az utasokat. A fejlődés robbanásszerű gyorsulását a szenzorok relatíve olcsóbbá válása, valamint a járművekbe építhető, nagy teljesítményű számítási egységek megjelenése nélkülözhetetlen volt. Ezen felül az adatfeldolgozás és a mélytanulás fejlődése is kulcsszerepet ezen fejlődés történetében mivel, ezek a technológiák teszik lehetővé, a járművek számára környezetük érzékelését.

Az önvezető autók iránti érdeklődést elsősorban a biztonság motiválja. A közúti balesetekben világszerte évente több mint egymillió ember veszti életét [1], és a balesetek döntő többsége emberi hibára – figyelmetlenségre, fáradtságra, rossz helyzetmegítélésre – vezethető vissza.

Egy gép nem fárad el, semmi sem vonja el a figyelmét, és valamint a reakcióideje optimális esetben rövidebb lehet az emberinél. Ez az eset azonban csak akkor valósulhat meg, ha a vezetést végző algoritmus minden helyzetben megbízhatóan, hatékonyan és biztonságosan működik. Nem elég, ha a jármű az esetek túlnyomó részében – egyenes úton, jó időben, kiszámítható hanem nehézkes körülmények között is helyt kell állnia. Ezen oknál fogva avalódi kihívást a ritka, váratlan helyzetek jelentik, például a forgalom dinamikája, egy sávba kerülő akadály vagy egy előzés közben megváltozó forgalmi helyzet. Egyetlen rosszul kezelt helyzet is súlyos balesethez vezethet. (észlelés, predikció, tervezés, szabályozás) bontva oldják meg. Ezek a rendszerek jól értelmezhetők, de minden szabályt és határesetet előre meg kell fogalmazni bennük, ami a valós forgalom sokszínűsége mellett rendkívül nehéz. Alternatívát a megerősítéses tanulás (*reinforcement learning*, RL) kínál, amelyben az ágens nem előre megírt szabályokat követ, hanem a környezettel való interakció során, jutalmak és büntetések alapján maga tanulja meg, hogyan kell cselekednie. Ez a megközelítés különösen alkalmas az olyan döntési feladatokra, mint a sávtartás, az akadályok kikerülése vagy az előzés, ahol a helyes viselkedést nehéz szabályokkal leírni, de könnyű megmondani, hogy egy adott kimenetel jó vagy rossz.

A vezetési feladatot hagyományosan egymásra épülő, alaposan tervezett modulokra bontják. Ezen blokkokat jól megtervezett logikai adatfolyam köti össze amelyet robotikai rendszerek kötnek össze (pl ROS2) [2]. Ilyen modulokra épülő önevezető rendszer például az Autoware Auto [3] amely egy ROS2 alapokra épülő robotikai rendszer önevezető autózáshoz. A rendszer kifinomultsága abban rejlik hogy minden felhasználó saját igényei szerint tudja formálni a rendszert és saját modulokat is adhat hozzá. Az ilyen bonyolult rendszerekkel szemben jelent egy lehetséges alternatívát a megerősítéses tanulás azonban ennek ára van. Az ágens próbálkozások és hibák sorozatán keresztül tanul amelyek költsége igen magas lenne a való világban, mivel

az elfogadható vezetési stratégia alakítása akár több százezer vagy akár több millió lépésen át is történhet. A tanulási folyamat során az ágensnek gyakorolnia kell és felfedeznie saját világát valamint korlátait. Ezt egy valós járművel elvégezni gyakorlatilag kivitelezhetetlen. Egy szenzorokkal felszerelt tesztjármű, a zárt tesztpálya, a biztonsági sofőr és a karbantartás költsége rendkívül magas, egy ütközés pedig nemcsak anyagi kárt okoz, hanem emberi életet is veszélyeztethet. Ráadásul az ágens szeretnénk felkészíteni a váratlan és veszélyes helyzetekre, hogy azokat is megfelelőképp kezelje és elkerülje a baleseteket.

Ezen okoknál fogva a tanítást és a tesztelést szimulált környezetben végeztem. A szimulációra a CARLA-t [4] egy nyílt forráskódú, kifejezetten önvezetés-kutatásra fejlesztett szimulátor, amely valósághű városi és autópálya-környezetet is képes egyaránt szimulálni valamint a fizikai jármű-modellt, amelyek elengedhetetlenek a reális valósághű szimulációkhoz.

Egy önvezető rendszer döntései csak annyira lehetnek jók, amennyire pontos a környezetéről alkotott képe, ez a kép pedig kizárólag a gépi látás és érzékelés révén a szenzorokból származik. A szenzor adatok feldolgozása az ágens számára igazi kihívást jelent. A komplex szenzorokból sokszínű és nagy varianciájú adat érkezik. Ilyen elgondolás okán a munkám során különböző szenzor adat előfeldolgozásokat végeztem ezzel optimalizálva a feladatokat az ágens számára.

Munkám során olyan ágens kifejlesztésén szeretnék dolgozni amely képes forgalmi szituációkat(pl előzés) is megfelelőképp biztonságosan kezelni, valamint betartja a KRESZ előírásokat is egyaránt. A fő gondolatot Terapaptommakol és tsai. [5] munkája adta, akik a CARLA szimulátorban DQN alapú ágenszt tanítottak, amely a LiDAR pontfelhő, a pozíció és a sebesség alapján fékezéssel és kitéréssel sikeresen elkerülte az úton parkoló járművet. Ez a munka azonban kizárólag statikus akadályra terjedt ki. Erre építkezve célom a dinamikus objektumok elkerülése, vagyis hogy az ágens a mozgó, lassabban haladó járműveket is biztonságosan, a KRESZ szabályait betartva kerülje ki.

2. Szimulációs környezet

A tanításhoz és a teszteléshez a CARLA (*Car Learning to Act*) szimulátorra [4, 6] esett a választásom. (1. ábra) A CARLA egy nyílt forráskódú UnrealEngine4 alapú szimulációs környezet, amelyet kifejezetten önvezető rendszerek fejlesztésére, tanítására, validálására és tesztelésére hoztak létre. A szimulátor magja C++ nyelven készült, a megjelenítést és a fizikai szimulációt pedig az Unreal Engine 4 (UE4) játékmotor végzi. Ennek köszönhetően a szimulátor valósághű, képet és járműdinamikát nyújt. A program képes a valóságban is gyakran használt önvezető szenzorok – többek között RGB-kamera, mélységkamera, szemantikus szegmentációs kamera, LiDAR, radar, GNSS és IMU – szimulált megfelelőit generálni minden forgalmi szituációban. Továbbá fel van készítve a dinamikus időjárás, napszak realisztikus kezelésére is. A forgalom pedig szabadon állítható és szabályozható. Így az ágens olyan típusú szenzoradatokról tanulhat, mint amilyeneket egy valódi jármű is szolgáltatna.

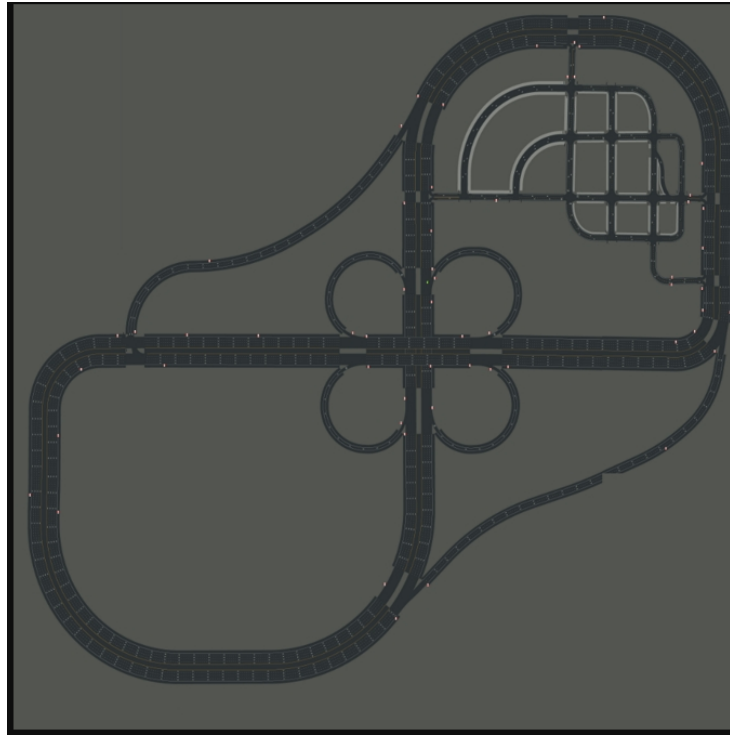


1. ábra. A CARLA szimulátor (forrás: [6])

A szimulátor kliens–szerver architektúrára épül [7], tehát a szerver futtatja magát a szimulációs környezetet ez által ő felel a világ és a járművek fizikájának számításáért, valamint a szenzoradatok előállításáért. Ez a kialakítás ugyanakkor teret enged annak is hogy a munka akár 2 számítógépen is megvalósuljon, például ez akkor indokolt ha nem elegendő egy számítógép grafikus feldolgozási erőforrása arra, hogy a szimuláció és az ágens tanulása is fusson egyaránt. A kliens program, melyet a felhasználó ír, hálózati interfészen (TCP kommunikációs protokoll segítségével alapértelmezetten a 2000-es porton) keresztül kérdezi le a szimuláció állapotát, illetve küld parancsokat a szervernek. Ilyenek lehetnek például a jármű kormány parancsa, gáz adagolása és fékezés is, ugyanakkor a kliens így kapja meg az egyes szimulálni kívánt szenzoradatokat is, a megfigyelni kívánt járműről. A kliensoldali API Pythonban is elérhető, ami nagyban megkönnyíti a fejlesztést. Munkám során a CARLA 0.9.16-os verzióját használtam szinkron módban, rögzített (20 Hz-es) szimulációs lépésközzel. Szinkron módban a szerver csak akkor lép a következő állapotba, amikor a kliens erre utasítja ez nagy segítségemre volt az ágens egyes lépéseinél, így a szenzoradatok és az ágens döntései mindig ugyanahhoz a szimulációs lépéshez tartoztak, valamint a tanítás is reprodukálható marad.

A CARLA-ban adott a lehetőség útvonaltervezésre is. A pálya úthálózata útpontokból (*waypoint*) álló topológiai gráfként érhető el, amelyen a szimulátorhoz tartozó GlobalRoutePlanner segítségével két tetszőleges pont között játszani könnyedséggel tervezhető [7] bármilyen kívánt útvonal. Ezt kihasználva minden epizód elején véletlenszerűen választott kezdő- és célpont között új, dinamikus útvonalat tervezek, amelyen az ágensnek végig kell haladnia ezzel kiküszöbölve azt, hogy esetlegesen rátanuljon az adott útvonalra, így segítem az általánosítási képességét.

A szimulátor alapértelmezetten tartalmaz több előre elkészített pályát (*Town*), amelyek variációja az átlagos önevezető feladatok többségét kielégíti. Munkám során ezek közül a Town04-et



2. ábra. A Town04 pálya felülnézeti térképe (forrás: [7])

választottam (2. ábra).

Választásom erre a pályára esett mivel az itt kialakított kisváros tökéletesen megfelel az elvárásaimnak. Alakja „8-as” alakú, végtelenített autópályával [7] megfelelő célom eléréséhez. A pálya egyszerre tartalmaz városi utcákat és többsávos autópályát, fel- és lehajtókkal, valamint lóhere alakú csomóponttal. Amint említettem célom az autópályás előzés megtanítása, ez a pálya a legalkalmasabb. A többsávos autópályán az ágens a lassabban haladó járművet a bal oldali sávba kihúzódva szabályosan meg tudja előzni, a körbeérő autópálya pedig hosszú, megszakítás nélküli vezetési szakaszokat tesz lehetővé.

3. Programozás és algoritmus

3.1. Megerősítéssel tanulás

3.2. Python programozási nyelv

3.3. Soft Actor-Critic SB3

4. Szensoradatok előfeldolgozása, Autóenkóderek

4.1. Kamera

4.2. LiDAR

4.3. Szenzorfúzió

5. Jutalom függvény

6. Betanult modellek összehasonlítása

7. Összegzés

Hivatkozások

- [1] World Health Organization. *Global Status Report on Road Safety 2023*. Geneva: World Health Organization, 2023. ISBN: 978-92-4-008651-7. URL: <https://www.who.int/publications/i/item/9789240086517> (elérés dátuma 2026. 09. 29.).
- [2] Steven Macenski és tsai. „Robot Operating System 2: Design, Architecture, and Uses in the Wild”. *Science Robotics* 7.66 (2022), eabm6074. doi: 10.1126/scirobotics.abm6074.
- [3] Shinpei Kato és tsai. „Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems”. *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. 2018, 287–296. old. doi: 10.1109/ICCPS.2018.00035.
- [4] Alexey Dosovitskiy és tsai. „CARLA: An Open Urban Driving Simulator”. *Proceedings of the 1st Annual Conference on Robot Learning*. 2017, 1–16. old.
- [5] Wasinee Terapattommakol és tsai. „Design of Obstacle Avoidance for Autonomous Vehicle Using Deep Q-Network and CARLA Simulator”. *World Electric Vehicle Journal* 13.12 (2022), 239. old. doi: 10.3390/wevj13120239.
- [6] CARLA Team. *CARLA: Open-source Simulator for Autonomous Driving Research*. URL: <https://carla.org> (elérés dátuma 2026. 10. 01.).
- [7] CARLA Team. *CARLA Simulator Documentation (0.9.16)*. URL: <https://carla.readthedocs.io/en/0.9.16/> (elérés dátuma 2026. 10. 01.).